GAME3023 Week 2 Notes

Unreal Engine: Development

Week 2 Notes • Arena Concept & Level Planning

Instructor: Joss Moo-Young • GAME3023 • Game Programming



Week 2

Arena Concept • Level Planning • Checkpoint 1

- Finalize the project direction before the blockout phase begins.
- Convert a broad game idea into a buildable combat arena plan.
- Review what Checkpoint 1 requires and how it will be assessed.
- Use arena-design thinking: scale, routes, flow, verticality, spawns, hazards, and risk/reward.
- Leave class ready to begin the two-week blockout phase in Unreal Engine.

Present complete or WIP pitches for Checkpoint 1

Semester Roadmap

The same project grows from concept to release.



Week 2 focus: prove that the concept is clear, buildable, and ready for blockout.

Checkpoint 1 is due this week. After today, the project shifts from planning into in-engine construction.

Part 1

What this week is about

Checkpoint 1 is the bridge between idea and buildable project.

This Week's Purpose

- Not to finish the level.
- Not to solve combat, networking, or AI yet.
- Not to write a huge Game Design Document.
- To make sure every project has a clear plan, achievable scope, and technical direction.
- To catch risky ideas before too much time is spent building them.

What Should Already Exist

- An Unreal Engine project created and able to open correctly.
- A project folder structure that will not become a mess by Alpha.
- An individual/team decision and basic workflow plan.
- An initial arena idea or theme.
- A rough list of gameplay mechanics, hazards, pickups, and opponent-path ideas.
- A record of technical risks or questions for Joss.

Checkpoint 1 Is Due This Week

- Checkpoint 1 is a planning and readiness assessment.
- It is worth 5% of the course grade.
- It should prove that the project is viable, not that the project is already built.
- Joss should be able to understand what you are making and why it is achievable.
- The plan can still change later, but it must be specific enough to guide blockout.

What Checkpoint 1 Should Prove

You are ready to build an arena, not just talk about a cool idea.

- You know the intended player experience.
- You have a physical space planned.
- You have considered routes, flow, scale, verticality, and spawn placement.
- You have identified interactive mechanics, hazards, pickups, and combat ideas.
- You have chosen or narrowed your opponent direction: AI or human/networked.

Checkpoint 1 Submission Snapshot

Component	What Joss Should See	Purpose
Concept	Short arena pitch and intended experience	Shows direction
Plan	Sketch or digital layout	Shows the space
Features	Mechanics, hazards, pickups, combat ideas	Shows technical intent
Opponents	AI or human/networked path	Shows implementation direction
Setup	Unreal project and workflow	Shows readiness

Checkpoint = meaningful progress and readiness. Milestone = required functional state.

Part 2

Checkpoint 1 details

Use this section as a final check before submission.

Required Submission Pieces

- Brief arena concept: theme, player experience, combat style, and project scope.
- A sketch, diagram, or digital layout of the proposed arena.
- Labels for major spaces, routes, spawn areas, verticality, pickups, hazards, and interactions.
- A preliminary technical feature list.
- Opponent direction: AI-controlled opponents or human/networked opponents.
- Unreal project created with a sensible folder structure.
- Team information and preliminary responsibility plan, if working in a team.

Concept Brief: Keep It Focused

- Explain the arena in one short paragraph or a few clear bullets.
- What is the setting or theme?
- What kind of movement/combat should the player experience?
- What makes the arena interesting beyond being a room?
- What is the realistic scope for an individual or team of up to three?

Arena Sketch or Digital Plan

- This can be hand-drawn, digital, diagrammed, or blocked using simple tools.
- It does not need to be beautiful.
- It must communicate playable spaces and layout decisions.
- Label major areas and intended routes.
- Show where players/opponents may spawn and where combat is likely to occur.

Your Plan Must Show

Design Element	What to Include
Main spaces	Central room, side areas, vertical platforms, corridors, open zones
Routes	Main route, alternate route, shortcut, loop, upper/lower path
Spawns	Initial player/opponent spawn ideas
Interactions	Doors, lifts, jump pads, switches, teleporters, moving elements
Hazards	Damage zones, fall risks, crushers, traps, dangerous movement
Pickups	Health, ammo, weapon, armour, temporary bonus, or equivalent

Opponent Direction

AI-Controlled Opponents

- Good for independent testing
- Uses AI Controller, NavMesh, Behavior Tree/State Tree, Perception
- Can be scoped as simple chase/attack behaviour
- Works well for online students working alone

Human / Networked Opponents

- Good for peer playtesting and LAN/lab play
- Uses Unreal multiplayer, replication, GameMode/GameState/PlayerState
- Must be kept simple and focused
- No accounts, matchmaking, back-end, or anti-cheat

Technical Feature List

- List features you expect to build later; do not build all of them now.
- Include at least several environmental mechanics and hazards you may use.
- Identify the combat style you are aiming for: projectile, hitscan, melee, ability-based, or other.
- Identify which features are required, optional, and risky.
- Show that your plan can be reduced if time becomes tight.

Scope Review

Too Small	Right Scope	Too Large
One hallway	One complete combat arena	Full campaign or open world
No hazards	Several meaningful hazards/mechanics	Huge physics sandbox
No opponent	AI or networked opponent path	AI + full networking + many modes
Unlabeled sketch	Clear plan with spaces/routes	Beautiful but unbuildable concept art
Vague idea	Specific, testable gameplay loop	Feature list with no priorities

Team Workflow Evidence

- If working in a team, list the team members.
- Identify initial responsibilities, but do not permanently lock people into roles.
- Decide how the team will communicate and track tasks.
- Decide how the Unreal project will be shared or backed up.
- Remember: every member should understand the overall project, not just one isolated piece.

Unreal Project Setup

- Project opens successfully.
- Main map/level exists or is ready to be created.
- Folder structure exists for maps, Blueprints, materials, UI, audio/VFX, characters, and documentation.
- Naming convention is understandable.
- Team members can access the project or understand the sharing/backup workflow.
- No critical setup problems should block Week 3 blockout work.

What Joss Should Be Able to See

A viable project direction that can start being built next week.

- What the arena is supposed to feel like.
- How the playable space is roughly organized.
- How players will move through the space.
- Where combat, hazards, pickups, and interactions might occur.
- Which opponent path the project is likely to use.
- What the team will work on during the blockout phase.

Common Checkpoint 1 Weaknesses

- Only describing the theme, with no actual playable layout.
- Drawing a beautiful map that does not show routes or scale.
- Planning too many mechanics before the arena exists.
- Trying to do both AI and networking without a reason.
- Choosing a theme that requires too much original art.
- Submitting an Unreal project that is not organized or not ready to build.

Part 3

Arena design overview

A good arena is a system of spaces, routes, decisions, and repeated encounters.

From Concept to Playable Space



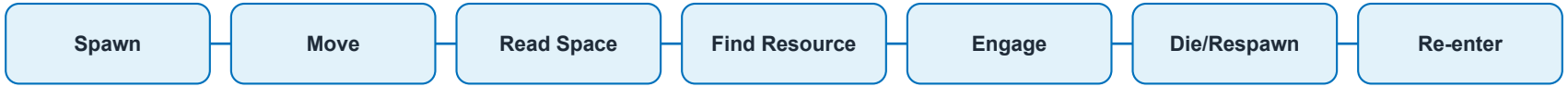
Do not jump straight from idea to finished visuals. The next phase is about proving the space works.

Week 2 plan → Week 3/4 blockout → Milestone 1 First Playable

Classic Arena Design Goals

- Players should find each other without feeling trapped in one corridor.
- There should be meaningful choices: chase, retreat, flank, take high ground, risk a shortcut.
- Important pickups or positions should create conflict.
- Hazards and interactive mechanics should shape movement or combat.
- Arena design should support repeated play, not a one-time walkthrough.

Arena Gameplay Loop



Your plan should explain where this loop might happen in the arena.

Alpha proves the loop works mechanically. Week 2 only needs to plan the space that will support it.

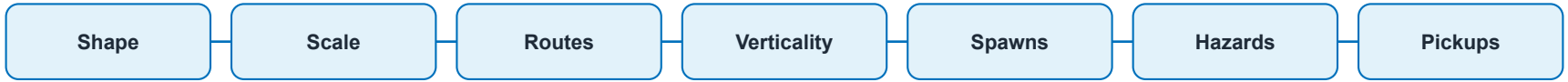
Space Categories

Space Type	Purpose	Example
Spawn area	Safe return to play	Protected alcove with multiple exits
Central conflict area	Main combat focus	Open platform, bridge, courtyard
Resource route	Encourages movement	Health/ammo/weapon path
Risk zone	Creates tension	Hazard floor, exposed shortcut
Vertical route	Adds tactical choice	Ramp, lift, upper platform
Recovery space	Lets player reset	Lower route, cover area

First-Pass Layout

- Start with a simple readable structure.
- Block major areas before adding small details.
- Ensure every major space has a gameplay purpose.
- Avoid making the first plan too symmetrical unless symmetry supports the intended gameplay.
- Think about how a player moves immediately after spawning.

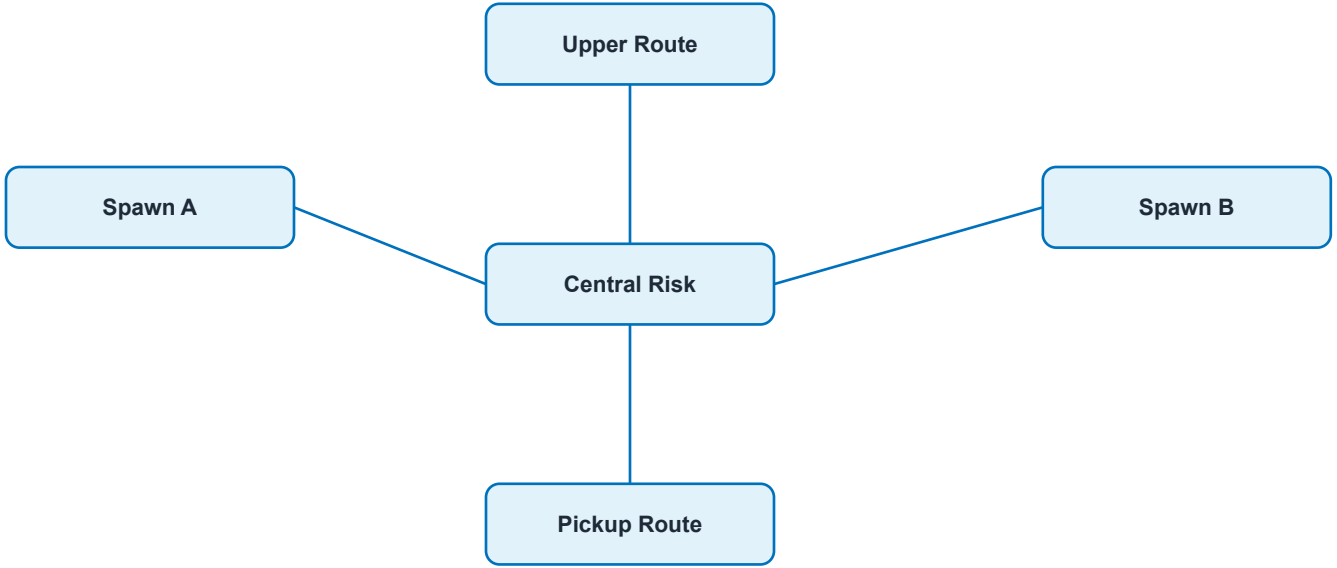
Design in Layers



Layering decisions helps prevent the plan from becoming a messy list of features.

- Shape: what are the major spaces?
- Scale: does it fit the player and combat style?
- Routes: how does the player circulate?
- Features: where do mechanics support the flow?

Example Top-Down Arena Plan



This is not a required layout. It shows the kind of labelled thinking expected in Checkpoint 1.

Routes

- A route is a meaningful path through the arena, not merely empty space between rooms.
- Main routes should be clear enough that players do not get lost.
- Alternate routes create tactical options: flank, retreat, avoid danger, reach high ground.
- Shortcuts are more interesting when they involve risk.
- Loops are usually better than dead ends in an arena.

Flow and Rotation

- Flow describes how players naturally move through the map over time.
- Good flow brings players back into conflict without making them feel forced.
- Think about where players go after respawning, winning a fight, or taking damage.
- Pickups, hazards, cover, elevation, and sightlines all influence flow.
- If nothing encourages movement, players may camp one location.

Sightlines

- Long sightlines support ranged combat and sniping.
- Short sightlines support ambushes, close combat, and fast encounters.
- Too many long sightlines can make spawning feel unfair.
- Too many blind corners can make combat feel random.
- Use walls, ramps, platforms, cover, and elevation to shape what players can see.

Verticality

- Height differences create tactical choices.
- Upper routes may offer visibility but expose the player.
- Lower routes may offer safety, cover, or faster movement.
- Ramps, stairs, lifts, jump pads, moving platforms, and ledges can all create vertical movement.
- Verticality should support gameplay, not just decoration.

Risk and Reward

- Good arenas make players choose between safety and advantage.
- A powerful pickup can sit in an exposed location.
- A shortcut can be faster but dangerous.
- High ground can provide visibility but make the player easier to target.
- Hazards are more interesting when players can intentionally use, avoid, or weaponize them.

Spawn Planning

- Spawn points should not place players directly into unavoidable danger.
- Spawns should avoid giving immediate unfair advantage.
- Players should have at least one reasonable route after spawning.
- In multiplayer projects, spawn fairness becomes especially important.
- In AI projects, bot/player spawn placement still shapes pacing and difficulty.

Pickups and Resource Placement

- Pickups are a reason to move through the arena.
- Health pickups can help a losing player recover.
- Powerful weapons or upgrades should usually involve risk.
- Ammo/resource placement can shape combat pacing.
- Place pickups where they support flow, conflict, or meaningful choices.

Hazards in the Plan

- Design hazards as gameplay features, not decorations.
- Players should be able to understand the hazard before or during use.
- Good hazards affect movement, positioning, timing, or combat choices.
- Examples: damage floor, crusher, falling zone, timed trap, moving obstacle, projectile hazard.
- Hazards should feel fair enough that players learn from mistakes.

Interactive Mechanics

- Interactive mechanics should change what players can do or how they move.
- Examples: doors, lifts, moving platforms, jump pads, teleporters, switches, timed bridges.
- Plan at least a few mechanics now, but expect to revise them after blockout.
- Ask: does this feature create a decision, shortcut, risk, surprise, or combat opportunity?
- Avoid adding interactions that look cool but do not affect play.

Theme Without Art Overload

- The theme should support level design and gameplay ideas.
- You can communicate theme with shapes, colour, lighting, material choices, names, and mechanics.
- Industrial: elevators, crushers, conveyors, warning lights, metal/concrete.
- Sci-fi: jump pads, teleporters, emissive paths, moving platforms.
- Dungeon: gates, traps, lava, narrow routes, reward rooms.
- Do not choose a theme that requires weeks of original 3D art.

Readability and Material Language

- Simple materials can tell players what spaces mean.
- Colour can distinguish team areas, danger areas, routes, or pickups.
- Lighting can guide players toward important locations.
- Contrasting materials can separate playable surfaces from background structure.
- Planning material language now makes Week 4 First Playable more readable.

Part 4

Preparing for blockout

The next two weeks are about proving the arena physically works in Unreal.

Blockout Mindset

Build the level quickly enough that you can test it early.

- Use simple shapes first.
- Test scale with the player character constantly.
- Focus on routes, elevation, collision, and readable spaces.
- Expect to revise the blockout after running through it.
- Do not decorate before the map is playable.

Scale Metrics

- Use the player character as the measuring tool.
- Check ceiling heights, door widths, platform widths, jump distances, ramps, and combat distance.
- If using a template character, test with that character from the start.
- Large arenas can feel empty; tiny arenas can feel chaotic.
- Test camera comfort, movement speed, collision, and weapon range together.

Use the Player Capsule Early



A plan that looks good from above may feel wrong once a player moves through it.

Next week, every important space should be tested by moving through it in Play-In-Editor.

Geometry Brushes and Modeling Mode

Geometry Brushes / BSP-Style

- Useful for fast blockout
- Simple additive/subtractive shapes
- Good for early walls, floors, ramps, volumes
- Old Unreal/UDK concept still useful as planning language

Modeling Mode

- Create/edit geometry inside Unreal
- Primitives, CubeGrid, extrude, edit surfaces
- Useful for cleaner blockout pieces
- Can assign materials and adjust geometry without external tools

Blocking Major Spaces

- Start with floors and boundaries.
- Add major rooms/zones next.
- Add vertical routes and main pathways.
- Add placeholder spawn areas and likely pickup/hazard locations.
- Keep small detail out until the First Playable layout feels good.

Collision and Navigation

- Walk every route in Play-In-Editor.
- Check whether the player catches on geometry.
- Check ramps, stairs, platforms, low ceilings, corners, and tight spaces.
- Make sure fall areas, hazard spaces, and boundaries are intentional.
- If using AI opponents later, begin thinking about navigable surfaces and NavMesh needs.

Folder and Naming Conventions

- Create clear folders before the project grows.
- Use names that describe purpose: BP_Door_Auto, BP_Hazard_DamageVolume, M_WarningStrip.
- Avoid generic names like NewBlueprint1 or FinalDoorReallyFinal.
- Group project assets logically: Maps, Blueprints, Materials, UI, Audio, VFX, Characters, Docs.
- Organized projects are easier to grade, debug, and present.

Testing in Editor

- Play-In-Editor should become part of every development session.
- Don’t wait until Milestone 1 to run through the arena.
- Test from different spawn locations.
- Test movement routes repeatedly.
- Write down problems immediately: scale, collision, dead ends, unfair sightlines, confusing paths.

Revision Planning

- Your Week 2 plan will not survive unchanged, and that is normal.
- Plan what you will test in Week 3.
- Decide what can change easily and what decisions may become expensive later.
- Keep a revision list: layout, scale, routes, spawn points, interactions, hazards, pickups.
- Revision is part of development, not evidence that the original plan failed.

Part 5

Opponent path planning

Do not build opponents this week. Make a realistic direction choice.

AI Path Planning

- AI opponents are a strong choice for independent testing.
- Plan where AI can spawn, navigate, detect, chase, attack, and recover.
- AI needs a navigable arena, sensible sightlines, and reliable combat interaction.
- Keep the first AI goal simple: move toward the player and participate in combat.
- Advanced behaviours can come later only if the core loop works.

Human / Networked Path Planning

- Human opponents are a strong choice for peer testing and classroom fun.
- Plan spawn fairness, routes, scoring, and repeated respawning from the beginning.
- Keep networking simple: player movement, combat, damage, death, respawn, score.
- Do not plan accounts, matchmaking, online services, or back-end features.
- You must be able to test the project reliably with available tools and machines.

Choosing the Opponent Path

Question	AI Opponents	Human / Networked
Can I test alone?	Very good	Possible with multiple instances, but more complex
Most familiar from prior Unreal courses?	Yes: AI/BT/EQS/Perception	Less direct from prior Unreal courses
Most fun for peer play?	Moderate	High
Main risk	AI behaviour feels weak	Replication/debugging becomes difficult
Best fit if…	You want reliable solo iteration	You want multiplayer testing and challenge

Avoid Over-Scoping Opponents

- You only need one opponent path.
- Do not plan both AI bots and networking unless there is a clear reason and time allows.
- One reliable, testable opponent solution is better than two broken ones.
- Plan the minimum opponent behaviour needed to make the arena playable.
- Alpha is where opponent functionality becomes required, not Week 2.

Part 6

Week 2 work session

Use class time to complete Checkpoint 1 and prepare for blockout.

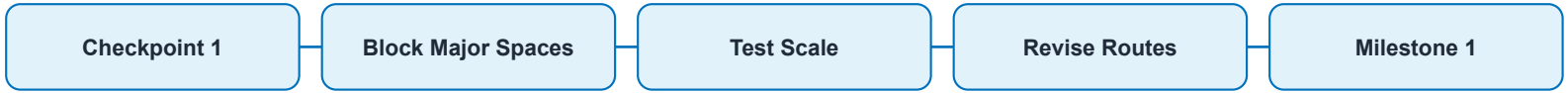
Week 2 In-Class Work Plan

- Review Checkpoint 1 requirements and compare them to your current plan.
- Complete or revise your concept brief.
- Improve your arena sketch/digital layout with labels and gameplay intent.
- Confirm individual/team workflow and responsibilities.
- Create or verify the Unreal project and folder structure.
- Discuss opponent path and main technical risks.
- Ask Joss scope questions before submitting.

Before You Submit Checkpoint 1

- Can someone understand the arena from your plan without you standing beside them?
- Does the plan show physical spaces, not just a theme?
- Does it include routes, spawns, verticality, hazards, pickups, and interactions?
- Have you chosen or narrowed the opponent path?
- Is the Unreal project ready to begin blockout next week?
- Is the scope reasonable for your team size?

Week 3 Starts the Blockout Phase



The next two weeks focus on turning the plan into an actual playable space.

Milestone 1 is due in Week 4: First Playable.

Recommended Reading

https://docs.google.com/document/d/1fAlf2MwEFTwePwzbP3try1H0aYa9kpVBHPBkylq-caY/edit

⚒ Structural geometry ⚒

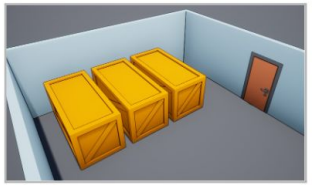
So now that you've started blocking out your level, the first step would be to just place out any of the larger more rigid surfaces and structural geometry your level has.
This means any buildings, caves, mountains, pits, etc. and not things like cars, benches or other non-rigid objects (often called "props").
This is because at first you just want to get the backbone of your level created, the thing that is less likely to change or move around than something like a car or a "decal".



Non-rigid "Props" on the left, rigid structural geometry on the right.

⚒ Form follows function ⚒

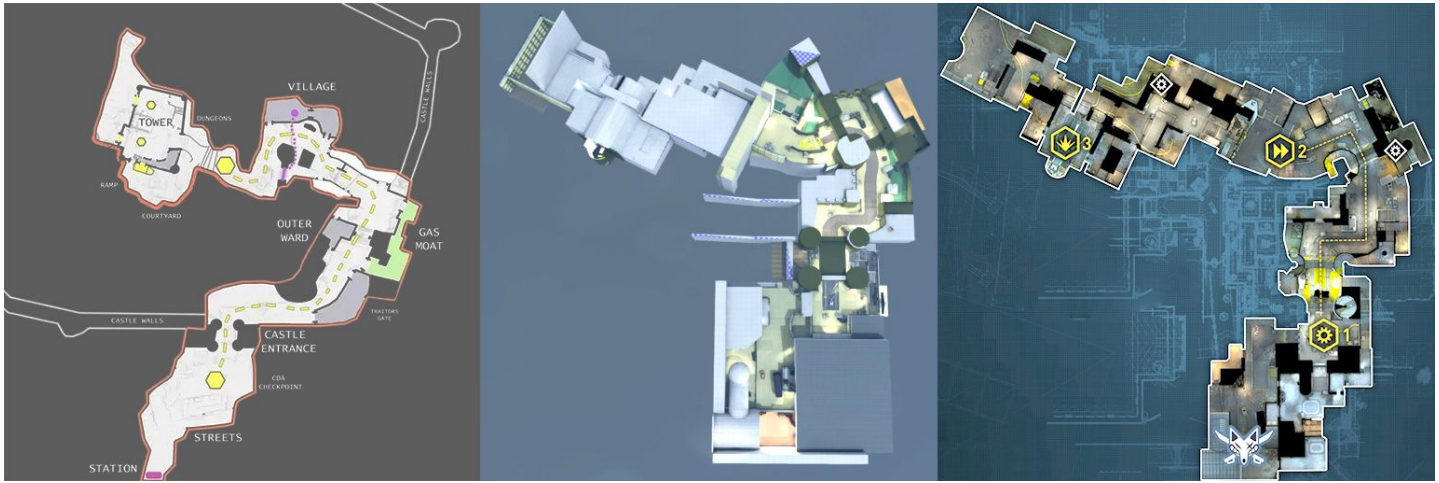
While we'll go into this in more detail in a later chapter, it's important to also consider how "form follows function" at this stage. That things in your level contextually makes sense.
A diner obviously has a bathroom, pipe systems shouldn't connect to nowhere and most buildings don't consist of a bunch of hallways.



How did these giant boxes get into this room with such a tiny door?

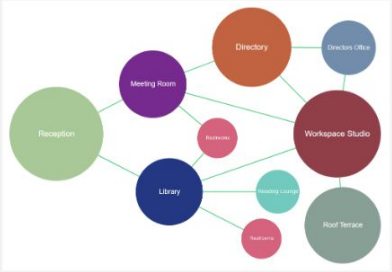
The Level Design Process

- **Premise/Concept:** Setting up a high-level vision
- **Sketch:** First explorations
- **Greybox:** Implementing and iterating on gameplay
- **Art implementing:** Dressing up your creation, maintaining readability
- **Final polish:** Bug fixing and final adjustments before release



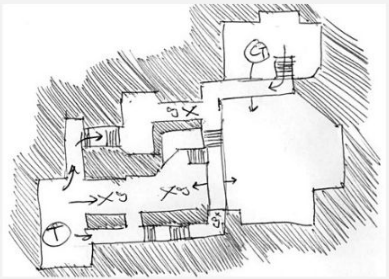
Level Sketch

1.Create a bubble diagram or flowchart



Basically, think of all the rooms/areas in your level, place them into ‘bubbles’ and connect them in the way they’ll be connected in the level.
The goal here is to get a sense for what areas your level will have, how they connect and how the player will flow through the level.
Here you could also loosely plan events like when the player meets an NPC, fights enemies or where they can find a key.

2.Draw up a map (not final) that roughly covers your entire level



The idea is to create a more detailed version of the bubble diagram that gives a more accurate image of how the entire level will look and connect.
Planned combat encounters, platforming sections, etc. should be fleshed out a bit and you should indicate roughly how many enemies and the like each room will have.
Here you can also write down some notes like “*The player will fight 1 wave of enemies and a turret, then 1 wave when the player has the turret*”.

Useful Documentation Links

Use these as starting points during this phase.

Unreal Editor Interface	Editor layout, viewport, Details, Outliner, Content Browser	Open documentation
Viewport Controls	Moving around the 3D viewport and manipulating objects	Open documentation
Modeling Mode	Creating and editing geometry directly in Unreal	Open documentation
Geometry Brush Actors	BSP/brush-style blockout concepts	Open documentation
Actors	Core objects placed in levels	Open documentation
Player Start Actor	Choosing where players begin or respawn	Open documentation

Common Week 2 Pitfalls

- Planning an arena that is too large for the semester.
- Submitting a theme instead of a playable layout.
- Not labeling the plan clearly enough.
- Choosing mechanics without explaining how they affect play.
- Leaving the opponent path completely undecided.
- Ignoring team workflow until the project becomes difficult to share.

Part 7

End of Week 2

Checkpoint 1 closes planning and opens blockout.

Main Takeaway

A strong plan makes the blockout faster, clearer, and easier to test.

- Your concept should be specific but flexible.
- Your layout should show gameplay intent.
- Your scope should match your team size and available time.
- Your opponent path should be realistic.
- Your Unreal project should be ready for construction.

Before Next Class

- Checkpoint 1 submitted.
- Arena plan revised based on feedback if needed.
- Unreal project ready for level construction.
- Team workflow confirmed.
- Documentation links bookmarked.
- Any major scope concerns discussed with Joss.
- Come ready to block out the arena in Unreal.

Week 2 End

- Next week: Arena Blockout — Part One.
- Focus: turning the plan into major in-engine spaces.
- Use Unreal Geometry/Modeling tools to establish scale, routes, and traversal.
- Milestone 1: First Playable is due Week 4.